

Les *Namespaces* en PHP

1. ? Qu'est-ce qu'un namespace ?

Un **namespace** (ou **espace de noms**) est un **contexte logique** qui permet d'**organiser le code** et d'**éviter les conflits de noms** entre classes, fonctions ou constantes.

Depuis PHP 5.3, les *namespaces* sont supportés nativement.

2. 🧠 Pourquoi utiliser des namespaces ?

Sans *namespace*, si deux bibliothèques déclarent une classe **User**, PHP lève une **erreur fatale**.

Avec les *namespaces*, vous pouvez avoir :

```
\App\Models\User  
\Lib\Auth\User
```

➡ Cela évite les conflits et améliore la lisibilité du code.

3. 🛠 Déclaration d'un namespace

Syntaxe :

```
<?php  
namespace MonProjet\Utils;  
  
function test() {  
    echo "Test utils";  
}
```

- Un seul namespace par fichier
 - Doit être la première ligne exécutée du fichier
-

4. 📁 Exemple concret

Fichier : **Maths/Addition.php**

```
<?php  
namespace Maths;  
  
function addition($a, $b) {
```

```
    return $a + $b;
}
```

Fichier : `index.php`

```
<?php
require 'Maths/Addition.php';

echo \Maths\addition(3, 4); // Affiche 7
```

5. L'instruction `use`

Pour **importer un namespace ou une classe** dans le code courant :

```
use Maths\addition;

echo addition(5, 2);
```

On évite de répéter `\Maths\addition` partout.

6. Exemple avec des classes

Fichier : `App/Models/User.php`

```
<?php
namespace App\Models;

class User {
    public function info() {
        return "Je suis un utilisateur";
    }
}
```

Fichier : `index.php`

```
<?php
require 'App/Models/User.php';

use App\Models\User;

$user = new User();
echo $user->info();
```

7. 🧩 Alias avec `as`

```
use App\Models\User as ModelUser;
use App\Controllers\User as ControllerUser;
```

8. 📖 Bonnes pratiques

- Correspondance PSR-4 : *Namespace* = structure de dossiers
- Un *namespace* par domaine logique (ex. `App\Models`, `App\Controllers`)
- Le nom racine (`App`) reflète le projet

9. ⚠️ À savoir

Point important	Explication
Les namespaces ne sont pas dynamiques	On ne peut pas les modifier à l'exécution
<code>use</code> ne fonctionne qu'en haut de fichier	Avant toute logique ou déclaration
Les classes sans namespace	Appartiennent au namespace global <code>\</code>

10. 📌 Résumé visuel

